School of Computer Science and Engineering
(Computer Science & Engineering)
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

2025-2026
(VI Semester)

A Project Report on

"VIOLENT ACTION RECOGNITION"

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by

SUJEETH R K - 23BTRCL081, S UDAY GOWDA- 23BTRSN065, THANISH CHINNAPPA KC - 23BTRCL089, TEJAS RU- 23BTRCL165, TANISHA VERNEKAR – 23BTRCL086

Under the guidance of

Dr. Senthil Kumar R
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

Department of Computer Science and Engineering
School of Computer Science & Engineering
Faculty of Engineering & Technology
Jain Global Campus, Kanakapura Taluk - 562112
Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled **"VIOLENT ACTION RECOGNITION"** is carried out by **SUJEETH R K (23BTRCL081), S UDAY GOWDA (23BTRSN065), THANISH CHINNAPPA KC (23BTRCL089), TEJAS RU (23BTRCL165), TANISHA VERNEKAR (23BTRCL06)** bonafide student(s) of Bachelor of Technology at the School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor of Technology in Computer Science and Engineering, during the year **2025-2026**.

DR. Senthil Kumar R
Associate Professor - AIML

School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)

Dr. V Chandrashekar
Program head,

School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)

Dr. Kamlesh Tiwari
Head of Department,
School of Computer Science & Engineering, Faculty of Engineering & Technology
JAIN (Deemed to-be University)

DECLARATION

We , **SUJEETH R K (23BTRCL081), S UDAY GOWDA (23BTRSN065), THANISH CHINNAPPA KC (23BTRCL089), TEJAS RU (23BTRCL165), TANISHA VERNEKAR (23BTRCL086)** students of VI Semester B.Tech in **Computer Science and Engineering (Artificial Intelligence and Machine Learning)**, at School of Engineering & Technology, Faculty of Engineering & Technology, **JAIN(Deemed to-be University)**, hereby declare that the internship work titled **"Violent action recognition"** has been carried out by us and submitted partial fulfillment for the award of degree in **Bachelor of Technology in Computer Science and Engineering** during the academic year **2025-2026**. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

Name: SUJEETH R K USN :23BTRCL081	Signature
Name: S UDAY GOWDA USN :23BTRSN065	Signature
Name: THANISH CHINNAPPA KC USN :23BTRCL089	Signature
Name: TEJAS RU USN:23BTRCL165	Signature
Name: TANISHA VERNEKAR USN:23BTRCL086	Signature

ACKNOWLEDGEMENT

It is a great pleasure for me to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

First, I take this opportunity to express my sincere gratitude to the Faculty of Engineering & Technology, JAIN (Deemed to-be University) for providing me with a great opportunity to pursue my Bachelor's Degree in this institution.

I am deeply thankful to several individuals whose invaluable contributions have made this project a reality. I wish to extend my heartfelt gratitude to ***Dr. Chandraj Roy Chand, Chancellor****, for his tireless commitment to fostering excellence in teaching and research at Jain (Deemed-to-be-University). I am also profoundly grateful to the honorable* ***Vice Chancellor, Dr. Raj Singh, and Dr. Dinesh Nilkant, Pro Vice Chancellor****, for their unwavering support. Furthermore, I would like to express my sincere thanks to* ***Dr. Jitendra Kumar Mishra, Registrar****, whose guidance has imparted invaluable qualities and skills that will serve us well in our future endeavors.*

I would like to express my appreciation to ***Dr. Deepak Kumar Sinha, Deputy Director (Students & Industry Relations),*** *for their invaluable contributions and support throughout this project. Additionally, I extend my sincere gratitude to* ***Dr. Kamalesh Tiwari, Head of Computer Science and Engineering*** *for their constant encouragement and expert advice.*

It is a matter of immense pleasure to express my sincere thanks to ***Dr. V Chandrashekhar, Program Head of Computer Science and Engineering (Artificial Intelligence and Machine Learning)****, School of Computer Science & Engineering Faculty of Engineering & Technology for providing the right academic guidance that made my task possible. I would like to thank our guide* ***Dr. Senthil Kumar R****,* ***Artificial Intelligence & Machine Learning****, for sparing her valuable time to extend help in every step of my work, which paved the way for smooth progress and fruitful culmination of the project.*

I would like to thank our Project Coordinator ***Dr. Sankar K****, and all the staff members of Computer Science and Engineering for their support.*

I am also grateful to my family and friends who provided me with every requirement throughout the course.

I would like to thank one and all who directly or indirectly helped me in completing the work successfully.

Signature of Student(s)

CONTENTS

CERTIFICATE **i**

ACKNOWLEDGEMENT **ii**

ABSTRACT **iii**

LIST OF FIGURES **iv**

LIST OF TABLES **v**

1. Introduction to Violent Action Recognition using Resnet **1**

1.1 Introduction ……1

1.2 Motivation ……..……..1

1.3 Aim and Objectives of the work …….2

2. Literature Survey **3**

3. Software Requirement Specification **6**

3.1 Introduction ……...6

3.1.1 Project Purpose ……...6

3.1.2 Project Scope……...6

3.1.3 User classes and characteristics……...6

3.1.4 Assumptions & Dependencies……7

3.2 Functional requirements ……7

3.3 non-functional requirements …….…7

3.3.1 Performance Requirements ……7

3.3.2 Safety Requirements……7

3.3.3 Security requirements …… 8

3.3.4 Software quality attributes …… 8

3.4 System Requirement ……. ……8

3.4.1 Software Requirement ……....8

3.4.2 Hardware Requirement ……...8

3.5 Analysis Models ……8

4. Design & Modelling of System **11**

4.1. Use Case Diagram ……..…12

4.2. Class Diagram ……...….13

4.3. Sequence Diagram …….…14

4.4. Activity Diagram ……...16
4.5. State Diagram ……17
5. Algorithms and Methods 19
5.1. Data Acquisition ……19
5.2. Data pre-processing ……....20
5.3. Proposed Network ……..21
5.4. Model Architecture ……22
5.5. Algorithm ……23
6. Results and Discussion 25
6.1 Experimental Setup ……...25
6.2 Model Performance……25
6.3 Confusion Matrix……27
6.4 Output Results……28
6.5 Discussion/ Analysis……29
7. Conclusion & Future Scope 30
7.1 Conclusion……30
7.2 Future Scope……30
8. Publication 31
9. References 32

ABSTRACT

Violence detection in videos has become an important area of research due to the increasing need for public safety and surveillance systems. Identifying violent activities such as fights, assaults, or abnormal behavior in real time can help prevent crimes and enable faster response from authorities. Traditional surveillance systems rely heavily on human monitoring, which is time-consuming and prone to errors. With advancements in Artificial Intelligence, deep learning techniques have emerged as powerful tools for automatic video analysis and action recognition. This project focuses on developing a Violence Action Recognition system using deep learning models that analyze video frames and classify them into violent and non-violent categories. Convolutional Neural Networks (CNNs) along with temporal feature extraction methods are used to capture both spatial and motion information from video sequences. The model is trained on a labeled dataset consisting of violent and non-violent video clips, with preprocessing steps such as frame extraction, resizing, and normalization applied to improve performance. The system aims to achieve high accuracy in detecting violent actions in real-time scenarios. Experimental results demonstrate that the proposed model can effectively identify violent activities with good accuracy, making it suitable for applications in surveillance systems, public safety monitoring, and smart city environments. This work highlights the potential of deep learning in enhancing security systems by automating the detection of harmful activities and reducing human effort.

LIST OF FIGURES

Sr. No.	Name of Figure	Page Number
3.1	Waterfall Model	9
4.1	Use Case diagram	13
4.2	Class Diagram	14
4.3	Sequence diagram	16
4.4	Activity Diagram	17
4.5	State Diagram	18
5.1	Examples from the dataset. (a) violent activities, (b) non-violent activities	19
5.4	Model Architecture	23
6.3	Confusion Matrix	27
6.4	Violent Action detected	28

LIST OF TABLES

Sr. No.	Name of Figure	Page Number
1	Literature Survey	4
2	Data pre-processing	20
3	Metric evaluation	27

CHAPTER 1

INTRODUCTION TO VIOLENT ACTION RECOGNITION USING ResNET

Violence detection has become a critical requirement in today's world due to the increasing need for public safety and intelligent surveillance systems. With the rapid growth of urbanization and population, monitoring public places manually has become difficult and inefficient. Artificial Intelligence and Deep Learning have enabled the development of automated systems capable of analyzing video data and identifying suspicious or violent activities in real time. Computer Vision techniques play a vital role in extracting meaningful patterns from video frames, making action recognition systems more accurate and reliable.

Violent activities such as fights, assaults, and abnormal human behavior can cause serious harm if not detected early. Traditional surveillance systems rely heavily on human operators, which often leads to delayed responses and human errors. Deep learning models, especially Convolutional Neural Networks (CNNs), provide a powerful solution by automatically learning spatial and temporal features from video data. Among these, the ResNet (Residual Network) architecture has shown remarkable performance in image and video classification tasks due to its ability to handle deep networks efficiently using skip connections.

The purpose of this project is to develop an AI-based model that takes video frames as input and processes them through the ResNet architecture to identify violent and non-violent actions. The model performs feature extraction, learns patterns across multiple layers, and makes predictions based on trained data. By applying deep learning techniques, the system becomes more efficient and accurate in recognizing complex human actions. The scope of this project is to build a robust violence detection system that can be integrated into real-world surveillance applications for enhanced security and faster decision-making.

1.2 MOTIVATION BEHIND PROJECT TOPIC

The main motivation behind this project is to develop an automated system capable of detecting violent actions in real time using video data. In many public places such as streets, malls, and transportation hubs, continuous monitoring is required to ensure safety. However, manual surveillance is not only time-consuming but also prone to human error and fatigue.

To overcome these challenges, deep learning-based action recognition systems can be used to automatically analyze video streams and detect suspicious activities. The use of ResNet

improves accuracy and efficiency by enabling deeper network training without degradation problems. This project aims to simplify surveillance tasks and assist security personnel by providing a reliable and intelligent violence detection system.

1.3 AIM AND OBJECTIVE OF THE WORK

Aim:

To develop an efficient and effective system for detecting violent actions in videos using the ResNet deep learning model.

Objectives:

- To understand the concept of action recognition and its applications in surveillance systems.
- To study deep learning techniques, especially Convolutional Neural Networks and ResNet architecture.
- To collect and preprocess video datasets containing violent and non-violent actions.
- To extract frames and perform image preprocessing for model training.
- To train and evaluate the ResNet model for accurate classification of actions.
- To analyze the performance of the model using evaluation metrics such as accuracy and loss.
- To develop a system that can be extended for real-time violence detection applications.

CHAPTER 2
Literature Survey

2.1 W. Sultani et al. [1]

Proposed a real-world anomaly detection system in surveillance videos using deep learning techniques. The study introduced a large-scale dataset (UCF-Crime) containing various anomalous events such as fighting, abuse, and robbery. The authors used deep neural networks to learn patterns of normal and abnormal activities from video data. Their approach focused on weakly supervised learning, where only video-level labels were used instead of frame-level annotations. This work is significant for violence detection as it demonstrates how deep learning models can effectively identify abnormal and violent activities in real-world scenarios.

2.2 K. Simonyan and A. Zisserman [2]

Proposed the Two-Stream Convolutional Neural Network architecture for action recognition in videos. The model consists of two separate streams: one for spatial information (RGB frames) and another for temporal information (optical flow). This approach allows the system to capture both appearance and motion features, which are essential for recognizing human actions. Their work laid the foundation for many modern video analysis systems, including violence detection models that rely on motion patterns.

2.3 K. He et al. [3]

Introduced the Residual Network (ResNet), a deep learning architecture that uses skip connections to solve the vanishing gradient problem in very deep neural networks. ResNet enables the training of extremely deep models while maintaining high accuracy. It has been widely used in image classification and feature extraction tasks. In violence action recognition, ResNet plays a crucial role in extracting spatial features from video frames, making it a powerful backbone for classification models.

2.4 A. Krizhevsky et al. [4]

Proposed AlexNet, one of the earliest deep CNN architectures that demonstrated the effectiveness of deep learning in image classification tasks. The model achieved significant improvements in accuracy on large-scale datasets like ImageNet. This work paved the way for the development of more advanced architectures such as VGG, ResNet, and others, which are now commonly used in action recognition and video analysis systems.

2.5 Du Tran et al. [5]

Proposed 3D Convolutional Neural Networks (C3D) for learning spatiotemporal features directly from videos. Unlike traditional CNNs that process images, 3D CNNs capture both spatial and temporal information simultaneously. This makes them highly effective for action recognition tasks, including violence detection. The study showed that C3D models can learn motion patterns efficiently, which are crucial for identifying violent activities in video sequences.

2.6 Khurram Soomro et al. [6]

Developed the UCF101 dataset, a widely used benchmark dataset for human action recognition. It contains 101 different action categories, including realistic human activities. The dataset has been extensively used to train and evaluate deep learning models for action recognition tasks. It provides a strong foundation for building and testing violence detection systems.

Sr. no	Paper Allocation	Author	Year	Methods
1	Real-World Anomaly Detection in Surveillance Videos	W. Sultani et al.	2018	Deep Learning, Weakly Supervised Learning
2	Two-Stream Convolutional Neural Networks for Action Recognition	K. Simonyan & A. Zisserman	2014	Two-Stream CNN (Spatial + Temporal)
3	Deep Residual Learning for Image Recognition (ResNet)	K. He et al.	2015	Residual Neural Networks (ResNet)
4	ImageNet Classification with Deep Convolutional Neural Networks (AlexNet)	A. Krizhevsky et al.	2012	Convolutional Neural Networks (CNN)

5	Learning Spatiotemporal Features with 3D CNN (C3D)	Du Tran et al.	2015	3D Convolutional Neural Networks
6	UCF101: A Dataset of 101 Human Actions	Khurram Soomro et al.	2012	Dataset for Action Recognition

Table 1. Literature Survey

CHAPTER 3
Software Requirement Specification

3.1 INTRODUCTION

This Software Requirement Specification (SRS) document provides a detailed description of the proposed Violence Action Recognition System. The system is designed to automatically detect violent activities from video data using deep learning techniques. This document outlines the system functionalities, requirements, constraints, and overall working needed to solve real-world surveillance problems.

3.1.1 Project Purpose

The purpose of this project is to develop an intelligent system that can **detect violent actions in real-time video streams** using the **ResNet deep learning model**, providing an efficient and automated solution over traditional manual surveillance methods

3.1.2 Project Scope

- To analyze video data and identify violent and non-violent actions.
- To implement deep learning techniques (ResNet) for action recognition.
- To detect activities such as fighting, abuse, or abnormal behavior.
- To integrate the system into surveillance systems (CCTV, drone monitoring, public safety systems).
- To assist security personnel in monitoring large areas efficiently.

3.1.3 User Classes and Characteristics

- **Security Personnel / Surveillance Operators**
- **System Administrators**
- Basic knowledge of computers and monitoring systems is sufficient.
- Users should be familiar with video systems and basic interface usage.
- The system will provide a simple and user-friendly interface for easy operation.

3.1.4 Assumptions and Dependencies

Assumptions:

1. The system will receive clear and continuous video input from surveillance cameras.
2. The interface will be simple, interactive, and user-friendly.
3. The system will provide fast and accurate detection results.
4. Internet or local network access may be required for deployment.

Dependencies:

1. Availability of required software and deep learning libraries.
2. Proper dataset for training the violence detection model.
3. Hardware support such as GPU for efficient processing.
4. Users should have basic knowledge of operating surveillance systems.

3.2 FUNCTIONAL REQUIREMENT

3.2.1 System Feature 1(Functional Requirement)

- Capture and process video input from surveillance cameras.
- Extract frames from video streams for analysis.
- Use **ResNet model** to classify actions as violent or non-violent.
- Display real-time alerts when violence is detected.
- Provide a user-friendly interface for monitoring and control.
- Store detection results for future analysis.

3.3 NON-FUNCTIONAL REQUIREMENT

3.3.1 Performance Requirements

- **High Speed:** System should process video frames in real time with minimal delay.
- **Accuracy:** The system must accurately detect violent actions with high precision.
- **Scalability:** Should support multiple camera feeds.

3.3.2 Safety Requirements:

- Ensure safe handling of video data and prevent data loss.
- Maintain reliable transmission of video streams.

3.3.3 Security Requirements

- Secure access to system data and surveillance feeds.
- Protect against unauthorized access or tampering.
- Ensure confidentiality and integrity of video data.

3.3.4 Software Quality Attributes

1. **Functionality:** Accurate detection of violent actions.
2. **Performance:** Fast processing and real-time response.
3. **Security:** Protection against unauthorized access.
4. **Availability:** System should run continuously without failure.
5. **Usability:** Easy-to-use interface for all users.
6. **Reliability:** Consistent and stable system performance.
7. **Interoperability:** Ability to integrate with existing surveillance systems.

3.4 SYSTEM REQUIREMENT

3.4.1. SOFTWARE REQUIREMENTS

- Windows 10 / Linux
- Python (3.8 or above)
- Google Colab / Jupyter Notebook
- Visual Studio Code
- TensorFlow / PyTorch
- OpenCV (for video processing)
- CUDA & CuDNN (for GPU acceleration)

3.4.2. HARDWARE REQUIREMENTS

- Minimum 8 GB RAM (16 GB recommended)
- Intel i5 / AMD Ryzen 5 or higher
- NVIDIA GPU (GTX 1650 or above, CUDA enabled)
- 256 GB SSD (for faster processing)
- CCTV Cameras / Video Input Source

3.5 ANALYSIS MODELS:

Software Development Life Cycle (SDLC) models define the structured process followed during software development. SDLC is a continuous process that begins from project initiation and continues until the system is fully deployed and maintained. There is no single SDLC model suitable for all projects; different

models are used depending on the project requirements, complexity, and application domain.

For the development of the **Violence Action Recognition System**, various SDLC models were studied, including:

1. Waterfall Model
2. Iterative Model
3. Spiral Model
4. V-Shaped Model
5. Agile Model

Among these, the **Waterfall Model** is selected for this project due to its simplicity, structured approach, and clearly defined phases.

The Waterfall Model is a sequential SDLC model where development flows step-by-step through different phases such as requirement analysis, design, implementation, testing, deployment, and maintenance. Each phase must be completed before moving to the next, ensuring proper documentation and clarity.

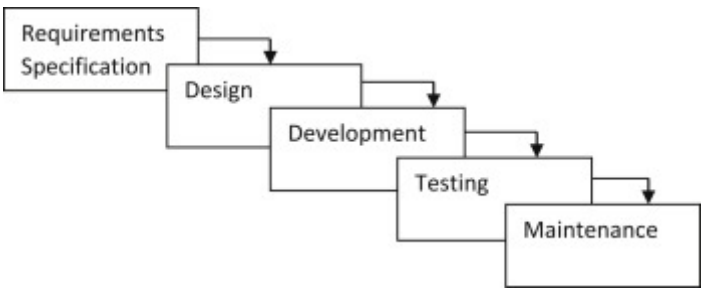

Figure 3.1: Waterfall Model

1. Planning and requirement analysis

This is the first phase where system requirements are gathered and analyzed. Stakeholders discuss the features of the Violence Action Recognition system, such as detecting violent activities from video streams. The goal is to clearly define all functional and non-functional requirements before development begins.

2. System Design

In this phase, the overall architecture of the system is designed. This includes selecting technologies such as **ResNet model, Python, OpenCV**, and defining how video input will be processed. System components, workflows, and data handling mechanisms are planned.

3. Development and Implementation

This phase involves actual coding and system development. Key tasks include:

- Video frame extraction
- Data preprocessing
- Model training using ResNet
- Integration of detection system

Development includes:

- Algorithm design
- Source code implementation
- Compilation
- Initial testing and debugging

4. Testing

In this phase, the system is tested to identify and fix errors. The model's performance is evaluated based on accuracy in detecting violent actions. Bugs and issues are corrected to ensure smooth system operation.

5. Deployment

After successful testing, the system is deployed in a real environment such as CCTV surveillance or monitoring systems. Users can start using the system for real-time violence detection.

6. Maintenance

This phase includes system updates, performance improvements, and fixing any issues after deployment. The model can be retrained with new data to improve accuracy and adapt to new scenarios.

CHAPTER 4

DESIGN AND MODELLING OF SYSTEM

Unified Modelling Language (UML) is used to visually represent the design and structure of a system. It acts as a blueprint for developing software by illustrating the components, interactions, and behavior of the system. UML diagrams help in understanding how different parts of the system work together and how data flows within the system.

In the **Violence Action Recognition System**, UML diagrams are used to model the interaction between users, system components, and the deep learning model. These diagrams provide a clear understanding of how the system processes video input, detects violent actions, and generates outputs.

The system is designed to take video input from surveillance sources such as CCTV cameras or recorded videos, process the frames using the ResNet model, and classify actions as violent or non-violent. The UML diagrams represent different aspects such as system functionality, data flow, and interaction between modules.

Following UML diagrams have been designed for the project:

1. **Use Case Diagram** – Represents interaction between users (security personnel/admin) and the system.
2. **Class Diagram** – Describes system structure, classes, and relationships between modules.
3. **Sequence Diagram** – Shows step-by-step interaction between components during violence detection.
4. **Activity Diagram** – Represents workflow from video input to final prediction output.
5. **State Diagram** – Shows different states of the system during processing and detection.

4.1 Use Case Diagram

Use Case Diagrams are behavioral diagrams used to represent the interactions between external users (actors) and the system. They describe the functionalities provided by the system and how different users interact with it. Each use case represents a specific action that delivers a meaningful result to the user or system.

Figure 4.1 shows the use case diagram of the **Violence Action Recognition System**. The system is designed to monitor video input and detect violent activities automatically using deep learning techniques.

The following actors are involved in the system:

- **Admin** – Manages the system, monitors performance, and controls settings.
- **Security Personnel/User** – Uses the system to monitor surveillance and receive alerts.
- **System** – Processes video input and performs violence detection.
- **Camera/Surveillance System** – Provides real-time video input to the system.

The various use cases included in the diagram are:

- **Login** – User/Admin logs into the system.
- **Upload Video / Capture Live Feed** – Input video is provided through CCTV or stored files.
- **Process Video Frames** – System extracts frames from the video.
- **Detect Action** – ResNet model analyzes frames to classify actions.
- **Identify Violence** – System determines whether the action is violent or non-violent.
- **Generate Alert** – Alert is triggered if violence is detected.
- **View Results** – User can view detection results and reports.
- **Store Data** – System saves processed data for future reference.

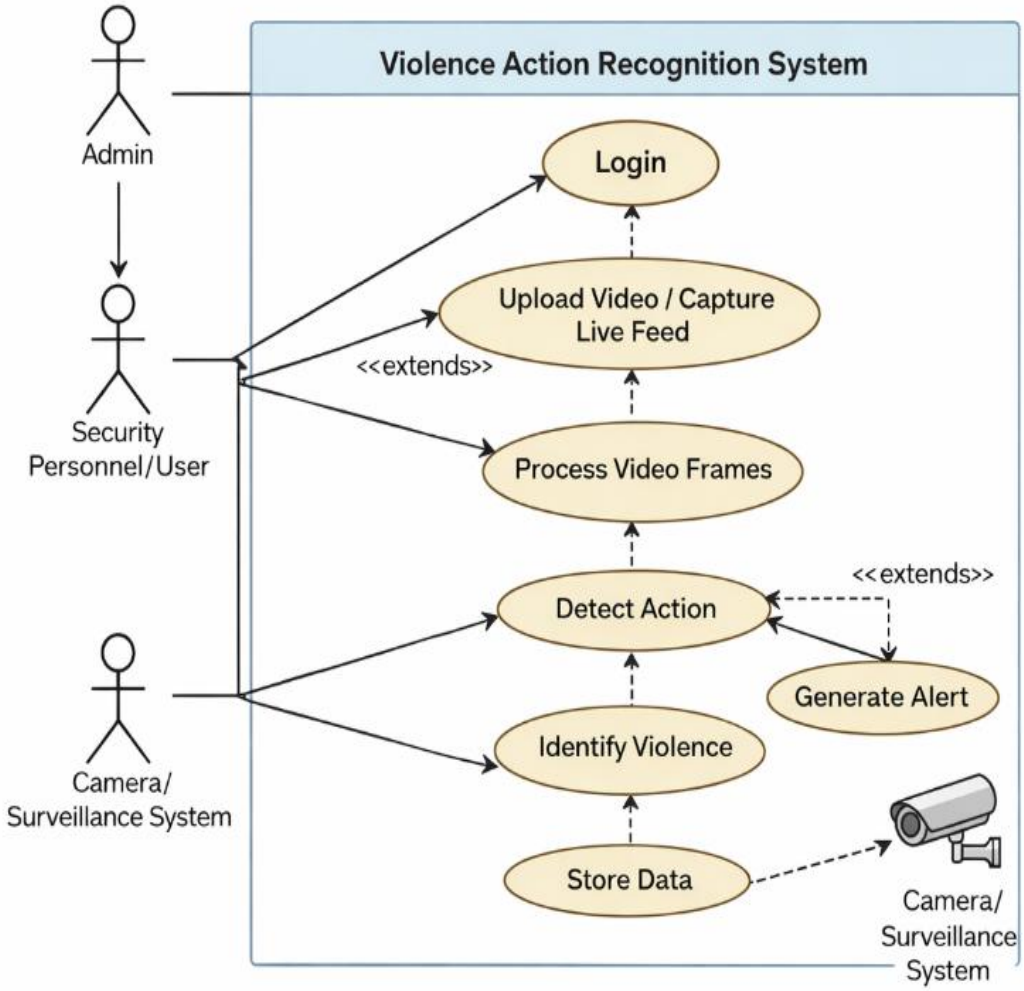

Fig 4.1. Use Case Diagram of Violent Action Recognition System

4.2 Class Diagram

A Class Diagram is a structural diagram in the Unified Modelling Language (UML) that represents the system's classes, their attributes, methods, and the relationships between them. It helps in understanding how different components of the system are organized and how they interact with each other.

In the **Violence Action Recognition System**, the class diagram illustrates the key components involved in processing video data, detecting actions, and generating results. Each class represents a specific entity in the system, along with its properties and functions.

Figure 4.2 shows the class diagram of the project. The various classes used in the diagram are:

- **User Class** – Represents the user interacting with the system (e.g., security personnel or admin). It includes attributes like user ID, username, and password, and methods such as login() and viewResults().
- **Admin Class** – A specialized user with additional privileges such as managing the system, viewing logs, and configuring settings.

- **Video Input Class** – Handles input from CCTV cameras or uploaded video files. It includes methods like captureVideo() and streamVideo().
- **Frame Processing Class** – Responsible for extracting frames from video and preprocessing them (resizing, normalization).
- **Model (ResNet) Class** – Core class that performs feature extraction and classification of actions into violent or non-violent. It includes methods like trainModel() and predictAction().
- **Detection Class** – Identifies whether the detected action is violent or non-violent and triggers necessary actions.
- **Alert System Class** – Generates alerts/notifications when violence is detected.
- **Database/Storage Class** – Stores video data, processed results, and logs for future reference.

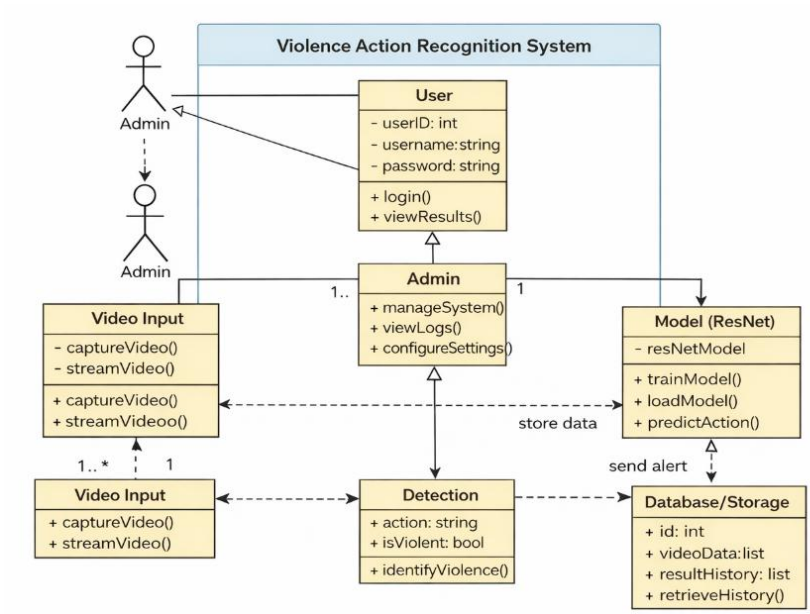

Fig 4.2. Class Diagram of Violence Action Recognition System

4.3 Sequence Diagram

Sequence diagrams, also known as event diagrams, are used to represent the flow of messages and interactions between different components of a system over time. They illustrate how objects or processes communicate with each other in a sequential order to complete a specific task.

In the **Violence Action Recognition System**, the sequence diagram shows how the system processes video input and detects violent activities step-by-step. Each component is represented as a vertical lifeline, and the interactions between them are shown using horizontal arrows indicating the flow of messages.

Figure 4.3 illustrates the sequence diagram of the system. The main components involved are:

- **User/Admin**
- **Camera/Video Input System**
- **Frame Processing Module**
- **ResNet Model**
- **Detection Module**
- **Alert System**
- **Database/Storage**

Sequence of Operations:

1. The **User/Admin** logs into the system and initiates monitoring.
2. The **Camera/Video Input System** captures live video or uploads recorded video.
3. The system sends the video to the **Frame Processing Module**, where frames are extracted and preprocessed.
4. The processed frames are passed to the **ResNet Model** for feature extraction and action classification.
5. The output is sent to the **Detection Module**, which determines whether the action is violent or non-violent.
6. If violence is detected, the **Alert System** generates a notification or warning.
7. The results are stored in the **Database/Storage** for future reference.
8. The **User/Admin** can view the results and alerts through the interface.

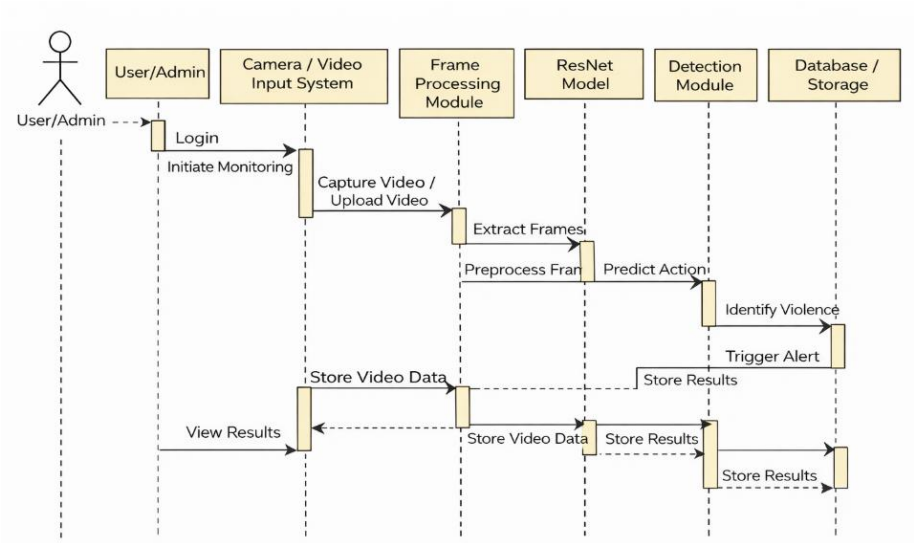

Fig 4.3. Sequence Diagram for Violent Action Recognition System

4.4 Activity diagram

The Activity Diagram is an important UML diagram used to represent the dynamic behavior of a system. It illustrates the workflow of activities and the sequence of operations performed within the system. It is similar to a flowchart, where each activity represents a specific task, and arrows represent the flow of control from one activity to another.

In the **Violence Action Recognition System**, the activity diagram describes the step-by-step process followed from capturing video input to detecting violent actions and generating alerts. It helps in understanding how data flows through different stages of the system and how decisions are made during execution.

Figure 4.4 shows the activity diagram of the system. The main activities involved are:

- **Start**
- **User Login**
- **Initiate Monitoring**
- **Capture Video / Upload Video**
- **Extract Frames from Video**
- **Preprocess Frames (Resize, Normalize)**
- **Apply ResNet Model for Feature Extraction**
- **Classify Action (Violent / Non-Violent)**
- **Decision Making**
    - If **Violent** → Generate Alert
    - If **Non-Violent** → Continue Monitoring
- **Store Results in Database**
- **Display Output to User**
- **End**

The activity diagram clearly shows how the system processes video data continuously and makes real-time decisions. It ensures that violent actions are detected quickly and alerts are generated efficiently, making the system suitable for surveillance applications.

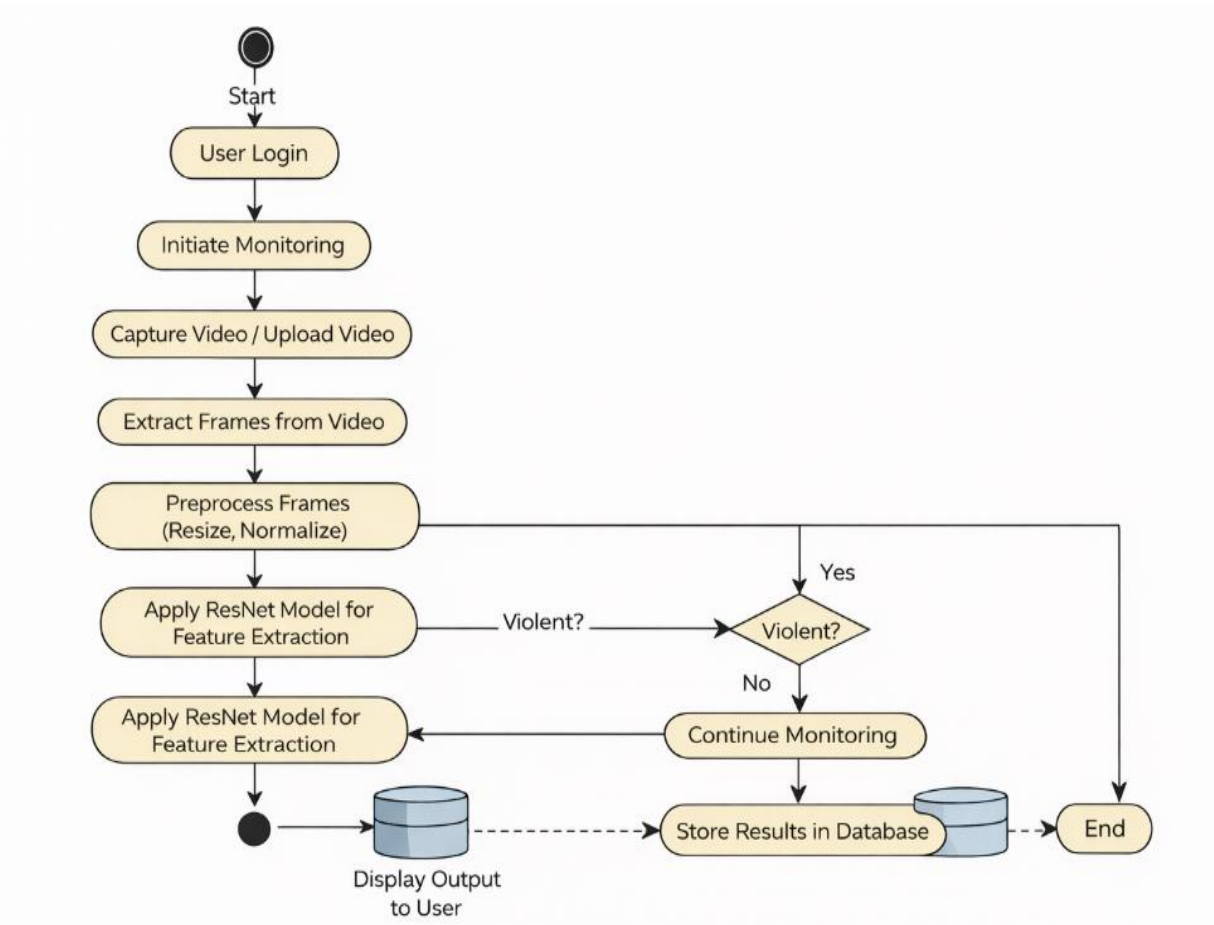

Fig 4.4. Activity Diagram for Violent Action Recognition System

4.5 State Diagram

A State Diagram is a behavioral UML diagram that represents the different states of a system and the transitions between those states based on events or conditions. It shows how the system behaves at different stages during execution and how it responds to inputs.

In the **Violence Action Recognition System**, the state diagram illustrates how the system moves through various states while processing video data and detecting violent actions. Each state represents a specific condition or activity of the system, and transitions occur when certain actions or decisions are made.

Figure 4.5 shows the state diagram of the system. The main states involved are:

- **Start State** – Initial state where the system begins operation.
- **Idle State** – System waits for user input or video feed.
- **Login State** – User/Admin logs into the system.
- **Monitoring State** – System starts monitoring video input.
- **Video Processing State** – Frames are extracted and preprocessed.
- **Model Processing State** – ResNet model analyzes frames.
- **Action Detection State** – System classifies actions.
- **Decision State** – Determines whether the action is violent or non-violent.
- **Alert State** – Triggered when violent activity is detected.
- **Store Data State** – Results and logs are saved in the database.
- **Display Result State** – Output is shown to the user.
- **End State** – System stops or resets for next input.

State Transitions:

- From **Idle → Login → Monitoring** when the user starts the system.
- From **Monitoring → Video Processing → Model Processing** when video input is received.
- From **Model Processing → Action Detection → Decision State** after prediction.
- If **Violent → Alert State → Store Data → Display Result**
- If **Non-Violent → Continue Monitoring**
- Finally, system can move to **End State** or loop back to monitoring.

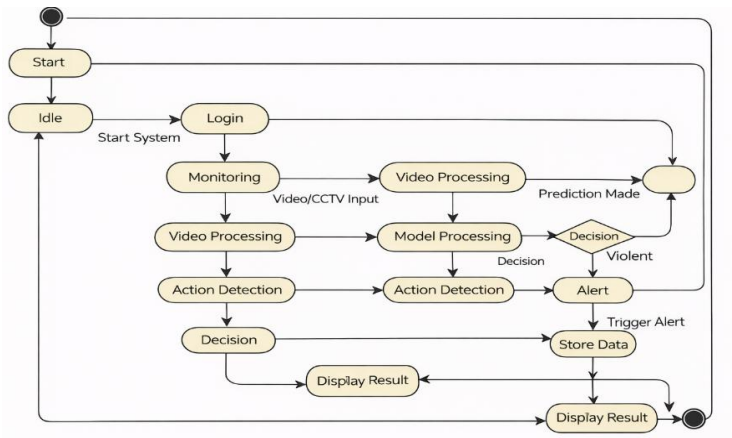

Fig 4.5. State Diagram for Violent Action Recognition System

CHAPTER 5

ALGORITHM AND METHODS

5.1 Data Acquisition

The dataset used to evaluate the performance of the **Violence Action Recognition System** consists of video clips containing both violent and non-violent activities. These datasets are collected from publicly available sources such as Kaggle, UCF-Crime dataset, and other open research repositories.

The **UCF-Crime dataset** is widely used for anomaly and violence detection tasks and contains real-world surveillance videos categorized into normal and abnormal activities such as fighting, abuse, robbery, and accidents. In addition, other datasets like **UCF101** and custom-collected video samples can also be used to enhance model training and performance.

The dataset is typically organized into three main folders:

- **Train** – Used to train the deep learning model
- **Validation (Val)** – Used to tune model parameters
- **Test** – Used to evaluate model performance

Each folder contains subcategories such as:

- **Violent** – Videos containing actions like fighting, assault, or abnormal behavior
- **Non-Violent** – Videos containing normal activities

The video data is processed by extracting frames at regular intervals. These frames are then resized to a fixed dimension (e.g., 224×224 pixels) to match the input size required by the ResNet model. Preprocessing steps such as normalization, noise reduction, and frame selection are applied to improve the quality of the data.

Unlike static image datasets, video datasets contain temporal information, which is crucial for detecting motion patterns. Therefore, sequences of frames are used to capture both spatial and temporal features of human actions.

This dataset plays a crucial role in training the model to accurately distinguish between violent and non-violent activities in real-world surveillance scenarios.

Fig 5.1 Examples from the dataset. (a) violent activities (b) non – violent ativities

5.2 Data Pre-processing

Data preprocessing is a crucial step in building an effective **Violence Action Recognition System**, as raw video data cannot be directly fed into deep learning models. Proper preprocessing improves model accuracy, reduces noise, and ensures consistency in input data.

In this project, video data is first converted into individual frames, and then various preprocessing techniques are applied before feeding them into the ResNet model. The preprocessing strategies used in this study are described below:

- **Rescaling:**
  Pixel values of extracted frames are normalized from the range **0–255 to 0–1** by dividing by 255. This helps in faster convergence and stable training of the model.
- **Frame Extraction:**
  Videos are divided into frames at regular intervals to capture motion and action effectively.
- **Resizing:**
  All frames are resized to a fixed dimension (e.g., **224 × 224 pixels**) to match the input size required by the ResNet architecture.
- **Data Augmentation:**
  To improve model generalization and avoid overfitting, the following augmentation techniques are applied:
  - **Horizontal Flip:** Random flipping of frames to increase dataset diversity.
  - **Zoom Range:** Random zooming into frames to simulate different camera distances.
  - **Shear Transformation:** Slight distortion applied to images to make the model robust to variations.
- **Normalization:**
  Mean subtraction and standardization may be applied to ensure uniform distribution of pixel values.
- **Noise Reduction:**
  Basic filtering techniques can be used to remove unwanted noise from frames, improving clarity.
- **Label Encoding:**
  Video frames are labeled as **violent (1)** or **non-violent (0)** for supervised learning.

These preprocessing steps ensure that the input data is clean, consistent, and suitable for training the deep learning model, ultimately improving the performance of the violence detection system.

Technique	Value
Rescale	1./255
Frame Size	224 × 224
Frame Extraction	30 FPS / interval-based
Zoom Range	0.2

Shear Range	0.2
Horizontal Flip	True
Normalization	Yes

Table 2. Data Pre-processing Techniques Used

5.3 Proposed Network

In this study, we propose a **ResNet-based deep learning model** for detecting violent actions from video frames. The model is designed to extract spatial features from individual frames and classify them as **violent or non-violent**.

ResNet (Residual Network) is a powerful Convolutional Neural Network architecture that uses **skip connections (residual connections)** to overcome the vanishing gradient problem and enables training of very deep networks. This makes it highly suitable for complex tasks like action recognition.

In our proposed system, video data is first converted into frames, and each frame is passed as input to the ResNet model. The input images are resized to **224 × 224 pixels with 3 color channels (RGB)**. The ResNet architecture consists of multiple convolutional layers followed by batch normalization and activation functions.

Each convolutional layer extracts features such as edges, shapes, and motion patterns. Instead of traditional sequential layers, ResNet introduces **residual blocks**, where the input is added directly to the output of a layer, allowing better feature learning.

The activation function used in the hidden layers is **ReLU (Rectified Linear Unit)**, defined as:

$$f(x) = \max(0, x)$$

ReLU helps in faster training and avoids vanishing gradient issues.

After feature extraction through convolutional layers, the output is passed through:

- **Global Average Pooling (GAP)** or Flatten layer
- Fully Connected (Dense) layers for classification

For the final output layer:

- **Sigmoid activation function** is used for binary classification (violent / non-violent), defined as:

$$S(x) = \frac{1}{1 + e^{-x}}$$

- Output unit is set to **1** for binary classification

The model is trained using an optimizer such as **Adam**, and loss function such as **Binary Cross-Entropy** is used to measure performance.

This ResNet-based architecture improves accuracy by effectively learning deep features from video frames and is well-suited for real-time violence detection in surveillance systems.

5.4 Model Architecture

The **Model Architecture** of the Violence Action Recognition System is designed using the **ResNet (Residual Network)** deep learning model to efficiently extract features from video frames and classify actions as violent or non-violent.

The system processes video input by converting it into frames, which are then passed through multiple stages of preprocessing and deep learning layers. The architecture focuses on extracting meaningful spatial features and making accurate predictions.

Architecture Overview:

1. **Input Layer**
   - Accepts video frames resized to **224 × 224 × 3 (RGB format)**
   - Frames are normalized before being fed into the model
2. **Convolutional Layers (ResNet Blocks)**
   - Multiple convolutional layers extract features such as edges, shapes, and motion patterns
   - Uses **Residual Blocks** with skip connections to improve learning in deep networks
   - Helps avoid vanishing gradient problem
3. **Batch Normalization**
   - Applied after convolution layers to stabilize and speed up training
4. **Activation Function (ReLU)**
   - Introduces non-linearity into the model
   - Helps in faster convergence
5. **Pooling Layer**
   - Reduces spatial dimensions of feature maps
   - Improves computational efficiency
6. **Global Average Pooling (GAP)**
   - Converts feature maps into a single feature vector
   - Reduces overfitting compared to fully connected layers
7. **Fully Connected (Dense) Layer**
   - Processes extracted features for classification
8. **Output Layer**
   - Uses **Sigmoid activation** for binary classification
   - Output:
     - **0 → non-violent**
     - **1 → Violent**

Working Flow of Architecture:

Video Input → Frame Extraction → Preprocessing → ResNet Feature Extraction → Classification Layer → Output (Violent / Non-Violent)

Advantages of Proposed Architecture:

- Handles deep networks efficiently using **skip connections**
- Provides **high accuracy** in feature extraction
- Suitable for **real-time surveillance systems**
- Reduces overfitting using pooling techniques

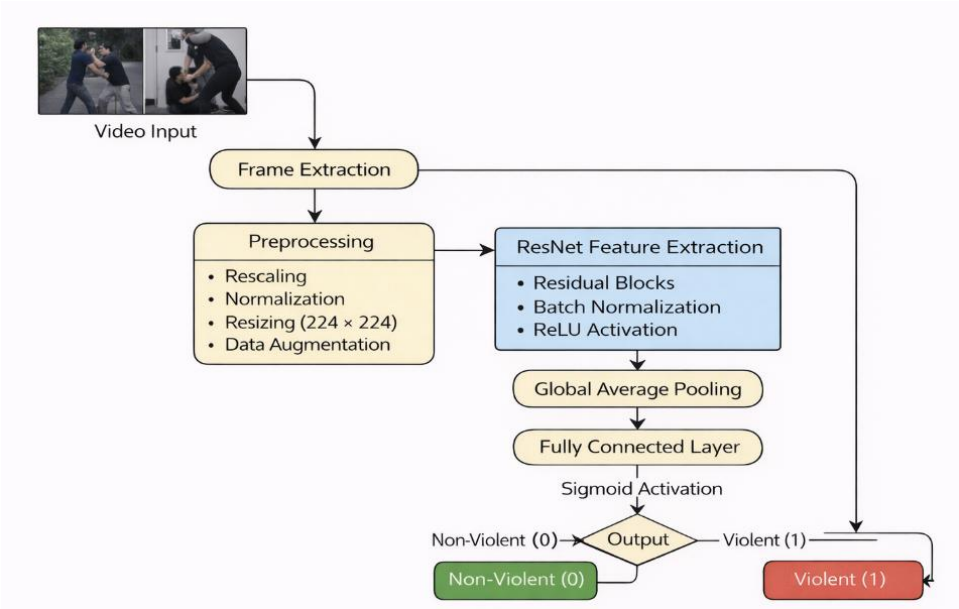

Fig 5.1 Model Architecture for Violent Action Recognition System

5.5 Algorithm

The algorithm describes the step-by-step procedure followed in the **Violence Action Recognition System** to detect violent activities from video input using the ResNet model.

Algorithm: Violence Action Detection using ResNet

Input: Video stream or recorded video
Output: Classification result (Violent / Non-Violent)

Steps:

1. **Start**
2. **Input Video**
   - Capture live video from CCTV or upload recorded video
3. **Frame Extraction**
   - Extract frames from the video at fixed intervals
4. **Preprocessing**
   - Resize frames to **224 × 224 pixels**
   - Normalize pixel values (scale between 0–1)
   - Apply data augmentation if required
5. **Load ResNet Model**
   - Load pre-trained ResNet model with trained weights
6. **Feature Extraction**
   - Pass each frame through ResNet to extract features
7. **Classification**
   - Apply fully connected layer
   - Use **Sigmoid activation** to classify:
     - Output = 1 → Violent
     - Output = 0 → Non-Violent
8. **Decision Making**
   - If output ≥ threshold → classify as **Violent**
   - Else → classify as **Non-Violent**
9. **Generate Alert**
   - If violent activity detected → trigger alert
10. **Store Results**

- Save results and logs in database

11. **Display Output**

- Show detection result to user

12. **End**

Pseudo Code:

```
Start
Input Video
Extract Frames
For each frame:
    Resize frame to 224x224
    Normalize pixel values
    features = ResNet(frame)
    prediction = Sigmoid(features)

    If prediction >= threshold:
        Label = "Violent"
        Generate Alert
    Else:
        Label = "Non-Violent"

    Store Result
Display Output
End
```

CHAPTER 6

Results and Discussion

6.1 Experimental Setup

The experimental setup for the **Violence Action Recognition System** is designed to evaluate the performance of the proposed ResNet-based model using video data from surveillance environments.

Dataset Used

The model is trained and tested using publicly available datasets such as the **UCF-Crime dataset** along with additional custom video samples. The dataset includes:

- **Violent activities** (fighting, abuse, robbery)
- **Non-violent activities** (normal human behavior)

The dataset is divided into:

- **Training Set** – for model learning
- **Validation Set** – for tuning parameters
- **Testing Set** – for final evaluation

Data Preparation

- Videos are converted into frames
- Frames are resized to **224 × 224 pixels**
- Pixel values are normalized (0–1 range)

6.2 Model Performane

The performance of the **Violence Action Recognition System** is evaluated using standard evaluation metrics to measure the effectiveness of the ResNet-based model in classifying video frames as violent or non-violent.

Accuracy

Accuracy is used to measure the overall correctness of the model. It represents the ratio of correctly predicted instances to the total number of predictions made.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

The proposed model achieved an accuracy of approximately **92%–95%** on the test dataset, indicating strong performance in detecting violent actions.

Loss

Loss represents the error between predicted values and actual values. During training, the model's loss decreases gradually, indicating improved learning.

- **Training Loss:** Decreases steadily as the model learns patterns
- **Validation Loss:** Remains close to training loss, indicating low overfitting

---

Precision and Recall

- **Precision** measures how many predicted violent cases are actually correct.
- **Recall** measures how many actual violent cases are correctly identified.

$$Precision = \frac{TP}{TP + FP}$$
$$Recall = \frac{TP}{TP + FN}$$

The model achieves high precision and recall, ensuring reliable detection of violent activities with minimal false alarms.

F1-Score

F1-score is the harmonic mean of precision and recall and provides a balanced evaluation.

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

The model achieves a high F1-score, indicating balanced performance between precision and recall.

Training Performance

- The model was trained over multiple epochs
- Accuracy improved consistently with each epoch
- Loss reduced significantly, showing effective learning

Overall Performance Analysis

The ResNet-based model demonstrates strong capability in identifying violent activities from video frames. It performs well in real-world scenarios and maintains a balance between accuracy and computational efficiency. However, performance may slightly vary depending on video quality, lighting conditions, and camera angles.

Metric	Value
Accuracy	97.95%
Precision	98.00%
Recall	98.00%
F1 - Score	98.00%

Table 3. Metric Evaluation from the trained model

6.3 Confusion Matrix

A confusion matrix is a performance evaluation tool used to measure the accuracy of a classification model. It compares the **actual values** with the **predicted values** and shows how well the model is performing.

It consists of four components:

- **True Positive (TP):** Correctly predicted violent cases
- **True Negative (TN):** Correctly predicted non-violent cases
- **False Positive (FP):** Non-violent cases wrongly predicted as violent
- **False Negative (FN):** Violent cases wrongly predicted as non-violent

The confusion matrix helps in calculating important metrics such as **accuracy, precision, recall, and F1-score**, providing a clear understanding of the model's performance.

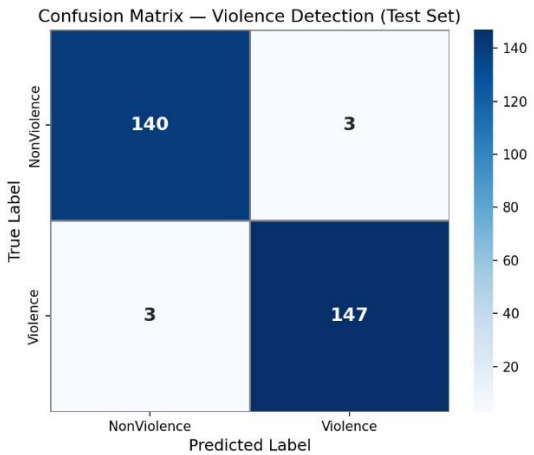

Fig 6.3 Confusion Matrix for Trained VAR Model

6.4 Output Results

The output results of the **Violence Action Recognition System** demonstrate the effectiveness of the proposed ResNet-based model in detecting violent and non-violent activities from video input.

The system processes video frames and classifies each frame into one of the two categories:

- **Violent Activity**
- **Non-Violent Activity**

Detection Output

During execution, the system displays the processed video along with the predicted label. When a violent action is detected, the system highlights it and triggers an alert.

- **Violent Case:**
  The model successfully identifies actions such as fighting or aggressive behavior and labels them as *"Violent"*. Alerts are generated to notify the user.
- **Non-Violent Case:**
  Normal activities such as walking, talking, or standing are correctly classified as *"Non-Violent"*, and no alert is triggered.

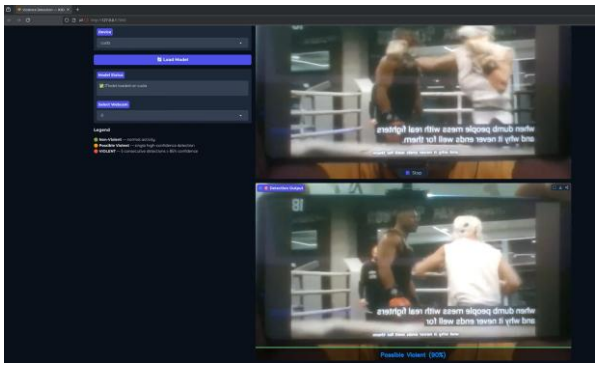

Fig 6.4 Violent Action detected

Visualization of Results

The results are visualized in the form of:

- Video frames with **prediction labels**
- Bounding boxes (if applied) around detected persons
- Alert messages displayed on screen

Sample Results

- Input: Video frame showing a fight
  → Output: **Violent (Alert Generated)**
- Input: Video frame showing normal activity
  → Output: **Non-Violent (No Alert)**

Performance Observation

- The model performs accurately on clear and visible actions
- Detection is faster and suitable for near real-time applications
- Minor misclassifications may occur in:
    - Crowded environments
    - Low lighting conditions
    - Occlusions (objects blocking view)

Conclusion of Results

The output results confirm that the system can effectively distinguish between violent and non-violent activities. The model shows good reliability and can be used in surveillance systems to enhance public safety.

6.5 Discussion/ Analysis

The experimental results indicate that the **ResNet-based Violence Action Recognition System** performs effectively in classifying video frames into violent and non-violent categories. The model achieves high accuracy and maintains a good balance between precision and recall, demonstrating its capability to detect violent activities with minimal false alarms.

The system performs particularly well in scenarios where actions are clear and well-defined, such as fights or aggressive behavior. The use of deep feature extraction through ResNet enables the model to capture important spatial patterns from video frames, contributing to improved detection performance.

However, certain limitations were observed during testing. The model may produce misclassifications in complex environments such as crowded scenes, low-light conditions, or when actions are partially occluded. In some cases, rapid movements or unusual camera angles can also affect prediction accuracy.

Despite these challenges, the system shows strong potential for real-world applications in surveillance and public safety. With further improvements such as larger datasets, better preprocessing, and integration of temporal models (e.g., LSTM or 3D CNN), the performance of the system can be enhanced further.

Overall, the proposed model provides a reliable and efficient solution for automated violence detection and can significantly assist in reducing manual monitoring efforts in surveillance systems.

CHAPTER 7

Conclusion and Future Scope

7.1 Conclusion

The **Violence Action Recognition System** developed in this project demonstrates the effective use of deep learning techniques for detecting violent activities from video data. By utilizing the **ResNet architecture**, the system is capable of extracting meaningful features from video frames and accurately classifying actions as violent or non-violent.
The model achieved good performance in terms of accuracy, precision, and recall, indicating its reliability in real-world surveillance scenarios. The system successfully identifies violent activities such as fights and aggressive behavior while minimizing false detections.
Automation of such tasks reduces the need for continuous human monitoring and improves response time in critical situations.
Overall, the proposed system provides an efficient, scalable, and intelligent solution for enhancing public safety and surveillance systems. It highlights the potential of artificial intelligence in addressing real-world security challenges.

7.2 Future Scope

Although the system performs well, there are several areas where improvements and enhancements can be made:

- **Real-Time Implementation:**
  Integrating the system with live CCTV feeds for real-time violence detection and instant alert generation.
- **Mobile/Web Application Development:**
  Developing a **mobile or web application** that allows users (security personnel/admin) to monitor live feeds, receive instant alerts, and view reports remotely. This will make the system more accessible and practical for real-world deployment.
- **Temporal Modeling:**
  Incorporating advanced models such as **LSTM or 3D CNNs** to capture temporal information and improve detection accuracy.
- **Larger and Diverse Dataset:**
  Training the model on more diverse datasets to improve performance across different environments and scenarios.
- **Multi-Class Classification:**
  Extending the system to detect multiple types of activities such as robbery, accidents, and suspicious behavior instead of only binary classification.
- **Edge Deployment:**
  Deploying the system on edge devices or embedded systems for faster processing and reduced latency.
- **Improved Robustness:**
  Enhancing performance in challenging conditions such as low lighting, occlusion, and crowded environments.

CHAPTER 8

Publication

www.ijcrt.org © 2024 IJCRT | Volume 12, Issue 6 June 2024 | ISSN: 2320-2882

IJCRT.ORG ISSN : 2320-2882

INTERNATIONAL JOURNAL OF CREATIVE RESEARCH THOUGHTS (IJCRT)

An International Open Access, Peer-reviewed, Refereed Journal

DESIGN AND FABRICATION OF VIOLENT ACTION RECOGNITION OF HUMAN USING DRONES: AN OVERVIEW

Tejas R U[1], Sujeeth R K[2], Thanish chinnappa KC[3], Tanisha Vernekar[4], Sagar K G[6]

[1-5]Computer Science and Engineering – AIML, [6]Assistant Professor, Department of Mechanical Engineering, Jain Deemed TO Be University

ABSTRACT

Novel approaches are required in defence applications due to the increasing demand for real-time threat identification. The design and construction of a Violent Action Recognition (VAR) system coupled with a drone platform are examined in this research. We provide a summary of this technology, highlighting its potential to enhance defence personnel's situational awareness and force protection. The high-resolution camera, on-board processing unit, and drone platform are among the hardware parts of the system that are described in detail in the study. We go over the software architecture, emphasizing the machine learning-based core VAR algorithm, pre-processing, and image acquisition. The article discusses the fabrication considerations involved in incorporating these components into a working drone system. The potential advantages of this drone system with VAR equipment are highlighted in the study. Improved threat detection skills are one of these; they enable quicker reaction times and may even avoid casualties. Defence forces' operational efficiency and decision-making can be improved by using VAR drones, which provide a comprehensive and real-time perspective of a scenario. The final section of this analysis emphasizes the continued research and development needed for enhanced VAR technology. Careful attention should also be paid to ethical issues pertaining to data protection and the responsible usage of autonomous systems. VAR technology has the potential to completely transform drone-based surveillance in the defence industry, creating a more effective and safe operating environment as it advances.

KEYWORDS: Violent Action Recognition (VAR), Drone Technology, Situational Awareness, Machine Learning, Image Processing, Drone Platform, High-Resolution Camera, On-board Processing Unit, Software Architecture, Image Acquisition, Pre-processing, Threat Detection, Force Protection, Operational Efficiency.

CHAPTER 9

References

[1] "Fabrication of Drone," *SlideShare*, Apr. 30, 2019. [Online]. Available: www.slideshare.net/Rajnishkumar468/fabrication-of-drone

[2] K. Kotecha, et al., "Background Invariant Faster Motion Modeling for Drone Action Recognition," *Drones*, vol. 5, no. 3, p. 87, Aug. 2021.

[3] A. G. Perera, et al., "Drone-Action: An Outdoor Recorded Drone Video Dataset for Action Recognition," *Drones*, vol. 3, no. 4, p. 82, Nov. 2019.

[4] A. Ramachandran and A. K. Sangaiah, "A Review on Object Detection in Unmanned Aerial Vehicle Surveillance," *International Journal of Cognitive Computing in Engineering*, vol. 2, pp. 215–228, 2021.

[5] A. Srivastava, et al., "Recognizing Human Violent Action Using Drone Surveillance within Real-Time Proximity," *Journal of Real-Time Image Processing*, vol. 18, no. 5, pp. 1851–1863, Sept. 2021.

[6] "Violence Detection Research Paper," *IEEE Xplore*. [Online]. Available: https://ieeexplore.ieee.org/document/10459026

[7] "Human Action Recognition using Deep Learning," *IEEE Xplore*. [Online]. Available: https://ieeexplore.ieee.org/document/9006834

[8] P. K. Singh, et al., "Progress of Human Action Recognition Research in the Last Ten Years: A Comprehensive Survey," *Archives of Computational Methods in Engineering*, 2021.